

Extracting Some Layers of Deep Neural Networks in the Hard-Label Setting

Isaac A. Canales-Martínez¹[0000–0002–0776–5060] and David Santos^{2*}

¹ Cryptography Research Center, Technology Innovation Institute
isaac.canales@tii.ae

² Instituto de Computação, Universidade Estadual de Campinas
d261032@dac.unicamp.br

Abstract. Although studied for several years now, parameter extraction of Deep Neural Networks (DNNs) has seen the major advances only in recent years. Carlini et al. (Crypto 2020) and Canales-Martínez et al. (Eurocrypt 2024) showed how to extract the parameters of ReLU-based DNNs efficiently (polynomial time and polynomial number of queries, as a function on the number of neurons) in the *raw-output* setting, i.e., when the attacker has access to the raw output of the DNN. On the other hand, the more realistic *hard-label* setting gives the attacker access only to the most likely label after the DNN’s raw output has been processed. Recently, Carlini et al. (Eurocrypt 2025) presented an efficient parameter extraction attack in the hard-label setting applicable to DNNs having a large number of parameters.

The work in Eurocrypt 2025 recovers the parameters of all layers except the output layer. The techniques presented there are not applicable to this layer due to its lack of ReLUs. In this work, we fill this gap and present a technique that allows recovery of the output layer. Additionally, we show parameter extraction methods that are more efficient when the DNN has contractive layers, i.e., when the number of neurons decreases in those layers. We successfully apply our methods to some networks trained on the CIFAR-10 dataset. Asymptotically, our methods have polynomial complexity in time and number of queries. Thus, a complete extraction attack combining the techniques by Carlini et al. and ours remains with polynomial complexity. Moreover, real execution time is decreased when attacking DNNs with the required contractive architecture.

Keywords: ReLU-Based Deep Neural Networks · Neural Network Extraction · Hard-label Attack.

1 Introduction

Artificial Intelligence (AI) has become a major topic of interest and discussion due to its increasing presence and impact in our technological environ-

* This work was conducted while David Santos was doing an internship at Technology Innovation Institute, UAE.

ment. In very simple terms, the goal of AI is to provide computational systems with processing capabilities generally linked with human intelligence. Machine Learning is the field of AI focused on making computational systems learn from large amounts of data to later perform computations on new data. Deep Neural Networks (DNNs) are prominent computing models within Deep Learning, a subfield of Machine Learning. DNNs have found several real-world applications and are typically used for classification, computer vision, language processing and medical diagnosis.

A Deep Neural Network is an artificial neural network consisting of multiple fully connected layers containing *neurons*. The input to a DNN is a vector of real numbers which is propagated through the layers of the network to produce the output vector. Each neuron performs a linear operation on its inputs followed by a non-linear operation called the *activation function*. An example of such function is the Rectified Linear Unit (ReLU). Before DNNs can perform meaningful computations, they must be trained. Training is the process in which the *parameters* of the network are initialised and iteratively adjusted in order to minimise the loss function on training data. The parameters are the *weights* and *bias* of each neuron in the network.

The parameters of a trained neural network may be a highly valuable asset. For instance, training a network can translate to great monetary costs if this process requires high computational time. Even when training is not as resource demanding (e.g., it can be done in polynomial time), the amount of available training data affects the quality of the resulting network. Thus, we may be interested in learning the parameters of a network if we do not have sufficient training data or computational resources to produce a comparable network.

Assume we have oracle access to a deep neural network, that is, we can query the network on an input of our choice and we receive the network's output as the response. Whether the parameters of this network can be extracted having exclusively oracle access is a long standing question. The effort to find an answer can be traced back to the 1990s. If we restrict our attention to DNNs using the ReLU as the activation function, recent results [7,5,6] show that it is indeed possible to extract the parameters. In the process, it was noted that extracting the parameters of ReLU-based DNNs resembles differential cryptanalysis on block ciphers. DNNs and block ciphers are comprised of layers and each layer consists of a linear operation followed by a non-linear one. In both cases, the linear operation computes on secret data (round keys in block ciphers and neuron parameters in DNNs) and the non-linear operation is publicly known (S-boxes and ReLUs, respectively). Indeed, the extraction attacks in [7,5,6] can be regarded as differential attacks.

Depending on the information provided by the oracle, an extraction attack has different levels of difficulty for the attacker. In [12], five different scenarios are identified. The least challenging one is when the attacker receives the raw output of the DNN (i.e., the vector of real numbers produced by the network); on the other hand, the most challenging scenario is the so-called *hard-label*,

where the attacker only receives the label of the most likely class. The complexity of extraction attacks is typically measured as a function of the total number of neurons in the network. Almost all attacks are framed in the easier raw-output scenario, and the work by Carlini et al. [7] and Canales-Martínez et al. [5] showed that parameter extraction in this setting can be done efficiently: the attack has polynomial complexity in both number of queries to the oracle and execution time. The most challenging hard-label setting is first addressed in [8] by Chen et al. Their attack is limited to DNNs with a low number of neurons and layers; although it requires polynomial number of queries, it has exponential time complexity. However, the recent work by Carlini et al. [6] introduces novel ideas yielding an attack in the hard-label scenario that enjoys polynomial time complexity while preserving polynomial query complexity.

1.1 State-of-the-Art Parameter Extraction Attacks on DNNs

The problem of DNN parameter extraction can be traced back to the initial work by Baum [1,2], Fefferman [10] and Blum and Rivest [3] over thirty years ago. This problem has received more attention in recent years and has motivated a great amount of research that considers different attack scenarios and security models. In [14], Oliynyk et al. present a comprehensive survey which shows the vast amount of work in the field. One of the main type of attacks against machine learning models are side-channel attacks. They exploit side-effects caused by the model during its execution; these side-effects could be arising from software or hardware. These attacks can allow recovery of the architecture and the parameters of the model. Another type of attacks are those where the attacker can obtain information from querying the model, but has no access to side-channel information.

In this work, we are interested in the security model where the attacker exclusively has oracle access to a ReLU-based DNN. Although there exists a great amount of literature on DNN parameter extraction, here we give a high-level overview of state-of-the-art attacks in the raw-output and hard-label settings. In Sections 2.2 and 2.3, we give a more detailed explanation which introduces only the ideas and notation required later in the manuscript.

The idea of the raw-output attack [7,5] is to recover the parameters of the first layer, “peel off” that layer, then recover the parameters of the second layer, peel off that layer and repeat until the last layer. Recovering the parameters comprises two steps: recover a multiple of the weights and biases, and recover the *sign* of each neuron in the layer, i.e., the sign of the multiplicative constant. To recover the parameters we employ *critical points*, which are inputs to the DNN where the slope of the output changes abruptly. These points provide highly valuable information: a neuron in the network takes value exactly zero before applying the activation function. If we collect enough critical points, we can recover, in polynomial time, (a multiple of) the parameters of the target layer by computing derivatives and solving a system of linear equations. Sign recovery as proposed by Carlini et al. [7] has exponential time complexity. Canales-Martínez et al. provide in [5] three sign-recovery techniques with

polynomial time complexity. In [11], Foerster et al. present an implementation combining the parameter recovery in [7] and sign recovery in [5]; the authors also propose improvements to reduce the real execution time of the attack.

Any ReLU-based DNN is a piecewise linear map. Critical points are exactly the points where the slope of the function changes. The set of critical points of a neuron form a *critical hyperplane*. These hyperplanes partition the input space into regions and the linear map is different in each region; Figure 1a shows an example of this. Crossing critical hyperplanes is the reason we observe the abrupt changes in the slope of the raw output of the network.

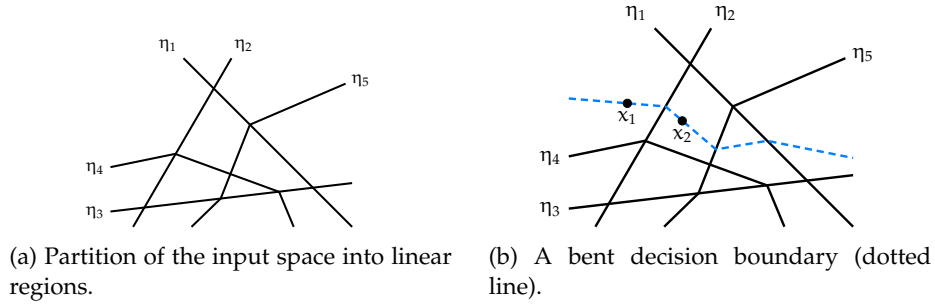


Fig. 1: Partition of the input space by critical hyperplanes. Hyperplanes of neurons in the second layer (η_4, η_5) are bent by hyperplanes of neurons in the first layer (η_1, η_2, η_3). Decision boundaries are bent by all critical hyperplanes. (Figure inspired from Figure 3 in [7] and Figure 2 in [6].)

In the hard-label setting, we lose access to the raw output, thus, we lose the ability to compute derivatives and detect critical points. The only noticeable event is the change in the output of the network from one class to another. An input where the network’s decision changes between classes is called a *transition point*. The set of transition points between two classes forms a hyperplane called *decision boundary*. Since the linear map changes between regions, a decision boundary “bends” when it crosses a critical hyperplane. Figure 1b depicts the partition of the input space together with a transition point x_1 and its decision boundary. Looking the figure, if walk from x_1 in a fixed direction along the decision boundary, we will eventually cross a critical hyperplane and we can find another point x_2 in the same decision boundary on the other side. The intersection of the bent decision boundaries is a subspace of the critical hyperplane, thus, we find critical points when we find this intersection. Repeating this procedure, we can recover the full description of the critical hyperplane.

The hard-label attack by Carlini et al. [6] follows the same strategy as the raw-output attack: recover the parameters of a layer, peel it off and repeat. Recovering the signature is equivalent to recovering the critical hyperplane using transition points with the procedure above. The sign is recovered using a metric relating the *state* of the target neuron with the speed of change in the

value of neurons in subsequent layers. This speed of change is captured by measuring distances in contiguous linear regions on both sides of the target neuron’s critical plane. The signature and sign recovery techniques in [6] require a polynomial number of queries and have polynomial time complexity. The attack recovers the parameters of all layers, except the output layer.

1.2 Our contributions

The output layer generates the raw output of the DNN and it typically does not have ReLUs. Due to this, the signature recovery technique in [6] is not applicable. Our main contribution is a method to extract the output layer in the hard-label setting. We exploit the fact that, in a transition point between two classes, the raw-output value of these classes is the same. This allows us to construct a linear equation where the variables are the parameters of the output layer. We can gather several linearly independent such equations and solve the system to recover the parameters. One caveat is that we cannot obtain a full-rank system. This is because there are several choices of values for the parameters in the output layer that yield an *equivalent* network. In particular, if the number of neurons in the last layer is r and the number of outputs in the DNN is n , the system has $n(r + 1)$ variables and the maximum number of linearly independent equations we can get is $n(r + 1) - r - 2$. Thus, to recover the last layer, we fix the value for $r + 2$ variables and solve the reduced full-rank system. The unique solution gives the value of the remaining parameters yielding an equivalent network.

Additionally, we show a hard-label version of two sign-recovery techniques from the raw-output attack. Namely, a hard-label version of the *freeze* method in [7] and the *SOE* method in [5]. We present a brief discussion on the conditions required for these methods to be applicable. As in the raw-output setting, SOE is preferable over freeze since it requires less calls to the oracle. Although these new hard-label techniques have limited applicability, the real time execution to recover some layers can be reduced.

We applied our new techniques to the CIFAR10 network used in [6]. We successfully recovered the output layer of that network. Execution time was over 6 hours in a non-parallelised execution and, in practice, all the time was spent finding transition points yielding the required number of linearly independent equations. The hard-label freeze and SOE methods are applicable to the first layer only and we successfully recovered the signs for that layer; in particular, SOE exclusively requires linear algebra operations and our experiments finished in under a second. To showcase the correctness of the sign-recovery techniques in deeper layers, we trained a new CIFAR10 network containing 1480 neurons spread across four layers. Our experiments with this network were also successful and we observed the same behaviour: SOE is practically instantaneous and the high execution time to recover the last layer (13 hours in this case) is due to the hardness of finding transition points generating the system of equations.

1.3 Organisation

In Section 2, we first present necessary definitions and introduce notation used in the rest of the manuscript; then, we give a more detailed explanation of the raw-output and hard-label parameter extraction attacks in [7,5] and [6], respectively. Section 3 discusses the new hard-label parameter extraction technique for the last layer, while the hard-label version of freeze and SOE are presented in Section 4. Our experimental results and conclusions are shown in Section 5.

2 Preliminaries

In this section, we first present necessary definitions and results used throughout the manuscript. These are standard in the literature (e.g., [7,5,8,6]) and we replicate them here keeping a consistent notation. Then, we give a description of the state of the art in parameter extraction attacks on deep neural networks.

2.1 Basic Definitions

Definition 1 (r-Deep Neural Network [7]). *An r-deep neural network f is a function that takes inputs from an input space $\mathcal{X}$ and returns values in an output space $\mathcal{Y}$. This function is composed as a sequence of functions alternating between affine layers f_i and a non-linear function σ :*

$$f = f_{r+1} \circ \sigma \circ f_r \circ \dots \circ \sigma \circ f_2 \circ \sigma \circ f_1. \quad (1)$$

We will only consider deep neural networks (DNNs) operating with real numbers, i.e., where $\mathcal{X} = \mathbb{R}^{d_0}$, $\mathcal{Y} = \mathbb{R}^{d_{r+1}}$ and d_0, d_{r+1} are positive integers.

Definition 2 (Fully Connected Layer [7]). *The i -th fully connected layer of a neural network is a function $f_i : \mathbb{R}^{d_{i-1}} \rightarrow \mathbb{R}^{d_i}$ given by the affine transformation*

$$f_i(\mathbf{x}) = A^{(i)}\mathbf{x} + \mathbf{b}^{(i)}, \quad (2)$$

where $A^{(i)} \in \mathbb{R}^{d_i \times d_{i-1}}$ is the weight matrix, $\mathbf{b}^{(i)} \in \mathbb{R}^{d_i}$ is the bias vector and d_{i-1}, d_i are positive integers.

Definition 3 (Neuron [7,5]). *A neuron is a function determined by the corresponding weight matrix, bias vector and activation function. Particularly, the j -th neuron of layer i is the function η given by*

$$\eta(\mathbf{x}) = \sigma \left(A_j^{(i)}\mathbf{x} + b_j^{(i)} \right),$$

where σ , $A_j^{(i)}$ and $b_j^{(i)}$ denote, respectively, the activation function, the j -th row of $A^{(i)}$ and the j -th coordinate of $\mathbf{b}^{(i)}$. An r-deep neural network has $N = \sum_{k=1}^r d_k$ neurons.

We will restrict the activation function σ to be the ReLU function, which is defined as $\sigma : x \mapsto \max(x, 0)$.

Definition 4 (Neural Network Architecture [7,5]). *The architecture of a fully connected neural network captures its structure: (a) the number of layers, (b) the dimension d_i of each layer $i = 1, \dots, r+1$ and (c) any additional constraints imposed on the weights and biases. We say that d_0 is the dimension of the input to the neural network and d_{r+1} denotes the number of outputs of the network.*

Definition 5 (Neural Network Parameters [7]). *The parameters θ of an r -deep neural network are the concrete assignments to the weights $A^{(i)}$ and biases $b^{(i)}$ for $i \in \{1, 2, \dots, r+1\}$.*

Definition 6 (Neuron State [5]). *Let $\mathcal{V}(\eta; x)$ denote the value that neuron η takes with $x \in \mathcal{X}$ before applying σ . If $\mathcal{V}(\eta; x) > 0$ then the state of η is active. Otherwise, the state of η is inactive³. The state of η on input x is denoted by $\mathcal{S}(\eta; x)$.*

Definition 7 (Linear Neighbourhood [5]). *The linear neighbourhood of $x \in \mathcal{X}$ is the set*

$$\{u \in \mathcal{X} \mid \mathcal{S}(\eta; x) = \mathcal{S}(\eta; u) \text{ for all neurons } \eta \text{ in the network}\}.$$

A deep neural network behaves as an affine transformation when its inputs are restricted to a linear neighbourhood. This is because no neuron toggles its state and the action of the ReLU can be seen as a constant. Let $F_{i,j} = \sigma \circ f_j \circ \dots \circ \sigma \circ f_i$, $1 \leq i \leq j \leq r$. Then, for all x in the same linear neighbourhood,

$$\begin{aligned} F_{i,j}(x) &= I^{(j)}(A^{(j)} \dots (I^{(i+1)}(A^{(i+1)}(I^{(i)}(A^{(i)}x + b^{(i)})) + b^{(i+1)}) \dots + b^{(j)}) \\ &= \Gamma x + \beta, \end{aligned}$$

where $I^{(\ell)}$ are diagonal matrices such that the k -th entry in the diagonal is 1 when neuron k in layer ℓ is active and 0 otherwise. That is, within a linear neighbourhood we can “collapse” contiguous layers into an affine transformation. Consequently, we can observe the change in the value of the layer- j neurons when we apply a change Δ to the input of the network, such that $x + \Delta$ is in the linear neighbourhood of x :

$$F_{i,j}(x + \Delta) - F_{i,j}(x) = \Gamma(x + \Delta) + \beta - (\Gamma(x) + \beta) = \Gamma\Delta. \quad (3)$$

Definition 8 (Critical point [7,5]). *A critical point for a neuron η is $x \in \mathcal{X}$ such that $\mathcal{V}(\eta; x) = 0$.*

When the DNN is evaluated at a critical point x for a neuron η , that neuron is at the boundary of toggling between the active and inactive states. Critical points define the boundaries of the different linear neighbourhoods the input

³ The authors in [7,5] consider an additional *critical* state when $\mathcal{V}(\eta; x)$ is exactly 0; we follow [8,6] and consider the critical state as a special case of the inactive state.

space $\mathcal{X}$ is partitioned into. These boundaries are called *critical hyperplanes* and each hyperplane is defined by one neuron in the network; see Figure 1a. Since one affine transformation corresponds to one linear neighbourhood, ReLU-based DNNs are piecewise linear functions.

Definition 9 (Hard-Label [8]). Let $f : \mathcal{X} \rightarrow \mathcal{Y}$ be an r -deep neural network, where $\mathcal{Y} = \mathbb{R}^{d_{r+1}}$. The hard-label on the output $f(x)$ is a label indicating the class with the largest probability when $f(x)$ is transformed into a probability vector. When $d_{r+1} = 1$, it is computed as

$$z(f(x)) = \begin{cases} 1, & \text{if } f(x) > 0, \\ 0, & \text{otherwise.} \end{cases}$$

When $d_{r+1} > 1$, $z(f(x))$ is the coordinate of $f(x)$ with the maximum value.

Definition 10 (Transition point [6]). A transition point is a point $x \in \mathcal{X}$ where the network's decision changes from one class to a different class, i.e., for all $\varepsilon > 0$ there exists $y \in \mathcal{X}$ with $d(y, x) < \varepsilon$ so that $z(f(y)) \neq z(f(x))$. Here, d denotes a distance in $\mathcal{X}$, for example $d(x, y) = \|y - x\|_2$ if $\mathcal{X} = \mathbb{R}^{d_0}$.

Definition 11 (Dual point [6]). A dual point is a point $x \in \mathcal{X}$ that is both a transition point and a critical point for some neuron η .

Transition points define hyperplanes, called *decision boundaries*, where the decision of the network changes between classes. Figure 1b depicts a decision boundary crossing critical hyperplanes. Since the affine transformation on each side of the critical hyperplane is different, the decision boundary is bent when crossing from one side to the other.

Definition 12 (Model parameter extraction attack [7]). Let f_θ denote a DNN with specific parameters θ , i.e., the values of its weights and biases. In a model parameter extraction attack, the attacker is given access to an oracle which, on query x , responds with $f_\theta(x)$. The objective of the attacker is to obtain a set of parameters $\hat{\theta}$ such that $f_{\hat{\theta}}(x)$ is as similar as possible to $f_\theta(x)$.

We will use the symbol $\hat{\cdot}$ to denote the recovered parameters. For example, $\hat{A}^{(i)}$ denotes the recovered weight matrix of layer i , whereas $A^{(i)}$ denotes the matrix of the original network.

2.2 Parameter Extraction Attack on DNNs - Raw-Output Setting

Carlini et al. introduced in [7] a model parameter extraction attack targeting ReLU-based DNNs. The idea is to recover the parameters of the first layer, “peel off” that layer and then repeat this process for the subsequent layers. Recovering the parameters of a layer consists of two steps: (1) recover the weights and biases, and (2) recover the *sign* of the neurons.

In the context of the attack, the DNN can be regarded as a composition of an *input function* F_{i-1} , the current target layer i and an *output function* G_{i+1} :

$$f = \underbrace{f_{r+1} \circ \sigma \circ \dots \circ \sigma \circ f_{i+1}}_{G_{i+1}} \circ \underbrace{\sigma \circ f_i}_{\text{layer } i} \circ \underbrace{\sigma \circ f_{i-1} \circ \dots \circ \sigma \circ f_1}_{F_{i-1}},$$

where F_0 is the identity map on the input when $i = 1$ and G_{r+2} is the identity map on the output when $i = r + 1$. The input function comprises the recovered layers while the output function comprises the non-recovered layers. The layers in those functions can be collapsed when the inputs x' to the network are restricted to be in the same linear neighbourhood of a point x :

$$F_{i-1}(x') = F_x^{(i-1)}x' + b_x^{(i-1)} \quad \text{and} \quad G_{i+1}(x') = G_x^{(i+1)}x' + b_x^{(i+1)}. \quad (4)$$

In order to perform the first step of the attack, we need to find critical points. Since any ReLU-based DNN f is a piecewise linear function, we have that $\frac{\partial^2 f}{\partial x^2}$ is 0 almost everywhere, except at those points. One method to find critical points, is to identify the discontinuities in the output of f when an input $x_1 \in \mathbb{R}^{d_0}$ is linearly transformed into another point $x_2 \in \mathbb{R}^{d_0}$. A detailed algorithm is given in [4]. By the coupon collector's argument and assuming uniformity, we expect to find multiple critical points for every neuron in the network by sampling enough pairs x_1, x_2 [7]. With critical points available, we can recover the parameters of the target layer. In particular, we recover a multiple of the weights of each neuron, called *signature*, and a multiple of the bias.

Definition 13 (Signature [6]). A signature of the j -th neuron in layer i is $\alpha A_j^{(i)}$, for some $\alpha \in \mathbb{R}$.

The signature $A_j^{(i)}/A_{j,1}^{(i)}$ of the j -th neuron in layer i , where $A_{j,1}^{(i)}$ is the first coordinate of $A_j^{(i)}$, can be recovered via partial second derivatives [7]. Obtaining signatures for first-layer neurons is a straightforward process, however, in deeper layers, we must account for the effect of neurons in other layers and process partial signatures; we refer to [7] for details on signature recovery. Using Equations (1) and (2) with the recovered signature $\hat{A}_j^{(i)}$ and the recovered previous layers, we obtain the bias $\hat{b}_j^{(i)} = b_j^{(i)}/A_{j,1}^{(i)}$ for the target neuron. Even though we do not recover the exact parameters from the target DNN, but a multiple of them, we can recover a network that is *equivalent* to the original network. Equivalent networks exist due to symmetries in them. For example, permuting the order of neurons or multiplying the weights and bias of a neuron by a positive constant are symmetries, whereas multiplying by a negative constant is not due to the nonlinearity of the ReLU. In our case, we recover $\hat{A}_j^{(i)} = \alpha A_j^{(i)}$ and $\hat{b}_j^{(i)} = \alpha b_j^{(i)}$, where $\alpha = 1/A_{j,1}^{(i)}$. Thus, before peeling-off a layer, we must determine the *sign* of each neuron, i.e., the sign of the multiplicative constant α .

Carlini et al. present in [7] a sign recovery method specific to the first layer of the network. We refer to this method as *freeze* and it is discussed in detail in Section 4.1. In [5], Canales-Martínez et al. show three improved sign extraction methods with polynomial time complexity and requiring polynomial number of queries. The first technique, called *SOE*, recovers simultaneously the sign of all neurons in a layer by solving a system of linear equations. This method is typically restricted to the first few layers in the network and is discussed in Section 4.3. The *neuron wiggle* technique is a general method that in principle is applicable to any layer in the DNN. The main idea is to maximise the change in the target neuron and observe the output of the DNN to decide whether the large change was blocked or not by the ReLU. The third technique is applicable only to the last layer of the network; this method also solves a system of linear equations and leverages the fact that, due to the lack of ReLUs in that layer, there is always a fixed output function. We refer to [5] for the details.

2.3 Parameter Extraction Attack on DNNs - Hard-Label Setting

In the raw-output setting, the attacker has access to the raw output of the network. This information is required to find critical points, where the behaviour of the DNN changes from one linear map to another. Critical points provide highly useful information regarding the value of the neurons, namely, they reveal that the input to the neuron’s ReLU is exactly zero at those points. In the hard-label setting, the attacker no longer receives the raw output of the DNN, only the label of the class with the highest probability. Therefore, it is not possible to detect critical points as before. The only event that can be exploited is passing through a transition point, i.e., observing that the hard-label output of the network changes from one label to another. This event, however, is not necessarily associated to a change in the internal state of the network (such as a ReLU toggle in the case of critical points).

The novel attack by Carlini et al. [6] finds dual points from transition points. Thus, the attacker regains access to critical points even though they are not readily accessible. The attack has polynomial complexity in both, number of queries and execution time, and follows the same strategy as in the raw-output setting: recover the parameters of the target layer, recover the sign of the neurons and repeat for the subsequent layers. In the remainder of this section, we briefly describe the main ideas behind the attack and refer to [6] for details.

The first step is to find the signature of neurons. The idea is depicted in Figure 2. We find a transition point x_1 between two classes and move along its $d - 1$ dimensional decision boundary in direction δ until we encounter a critical hyperplane. When we move in the same direction and cross the critical hyperplane, we end up in a point outside the decision boundary (because it was bent by the critical hyperplane). We project this point onto the decision boundary on the other side of the critical hyperplane to get another transition point x_2 . By carefully choosing inputs in the vicinity of x_1 and x_2 , we can obtain the normal vectors m_{off} and m_{on} of the decision boundary on each side of the critical hyperplane. The normal vectors give a complete definition

of the decision boundaries and allow us to compute the $d - 2$ dimensional subspace D of dual points where they intersect. This subspace also gives us a partial definition of the $d - 1$ dimensional critical hyperplane, since the former is a subspace of the latter. To recover the missing dimension, we repeat the same procedure in a different location of the input space and, with very high probability, we find a different subspace of dual points D' for the same neuron (i.e., critical hyperplane). Then, we obtain the full description of the critical hyperplane with D and D' which in turn yields the signature of the neuron.

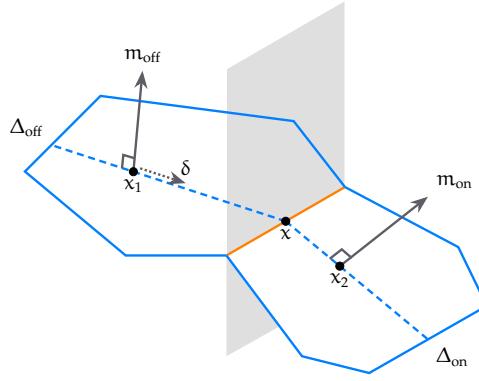


Fig. 2: Graphical representation of the procedure to find a space of dual points D . The distances Δ_{off} and Δ_{on} are used to recover the sign of the target neuron. (Figure inspired from Figure 4 in [6].)

To recover the sign, Carlini et al. employ a measurement strongly correlated to the speed of change in the value of neurons. The main idea is also depicted in Figure 2. Starting from a dual point x , we move along the decision boundary in a direction that is perpendicular to the subspace of dual points D until we find the hyperplane of another neuron. We do this traversal on both sides of the target neuron's critical hyperplane and measure the distances Δ_{on} and Δ_{off} . The speed of change in inputs for neurons in future layers is affected by the status of the current target neuron. Then, in most cases, we expect linear neighbourhoods on the positive side of the ReLU to be narrower compared to those on the negative side, in other words, $\Delta_{\text{on}} < \Delta_{\text{off}}$. This property is used to make a decision on the sign of the neuron. However, to overcome making wrong decisions, we decide the sign of the neuron after testing a sufficiently large number of dual points.

The one piece missing in the attack by Carlini et al. is the extraction of the output layer. This layer computes the output of the neural network before the hard-label function is applied. It is not possible to recover this layer with the techniques in [6] due to the lack of ReLUs (see Definition 1). In Section 3 we present a method to recover the output layer in the hard-label setting.

Additionally, in Section 4, we show how to recover the sign of neurons in layers that are *sufficiently contractive* more efficiently. Our methods require finding transition points which can be done with the algorithm in [6].

3 Extracting the Output Layer

In this section, we show how to recover the output layer. Section 3.1 first discusses how equivalent networks can be generated by varying the parameters in that layer. Then, in Section 3.2, we describe a technique applicable for networks with one output, also called binary classifiers, while networks with multiple outputs are addressed in Section 3.3.

3.1 Equivalent Deep Neural Networks

The output layer has no ReLUs and is defined by $A^{(r+1)}$ and $b^{(r+1)}$. Let

$$f_{1..i}(x) = \sigma \circ f_i \circ \dots \circ \sigma \circ f_1(x)$$

be the function computing the output of the i -th layer, where $1 \leq i \leq r$, and recall that a hard-label function z is applied to the output of the DNN. Then, with input x , the hard-label output of a DNN is

$$z(A^{(r+1)}y^{(r)} + b^{(r+1)}),$$

where $y^{(r)} = f_{1..r}(x)$ is the output of layer r . In the rest of this section, we will simply write A , b and y instead of $A^{(r+1)}$, $b^{(r+1)}$ and $y^{(r)}$, respectively.

The hard-label function first transforms the raw output of the DNN into a probability vector and then returns the label with the highest probability. We will assume that the function computing the probability vector (i) is invariant under translation by a vector with the same value in all its coordinates and (ii) assigns larger probability values to larger coordinates in the input vector. An example of such a function is *softmax*. This function is very frequently used in neural networks and we will use it as the default choice. On input $u \in \mathbb{R}^n$, softmax produces a vector $s(u) \in (0, 1)^n$ as follows:

$$s(u)_i = \frac{e^{u_i}}{\sum_{j=1}^n e^{u_j}}.$$

Let $\bar{c} \in \mathbb{R}^n$ be a vector with all coordinates having the same value c . Softmax is invariant under translation by $\bar{c}$:

$$s(u + \bar{c})_i = \frac{e^{u_i + c}}{\sum_{j=1}^n e^{u_j + c}} = \frac{e^{u_i} e^c}{e^c \sum_{j=1}^n e^{u_j}} = s(u)_i.$$

Even though softmax is not invariant under scaling (i.e., $s(u) \neq s(c \cdot u)$ for $c \in \mathbb{R}$), larger coordinates in u will remain larger when scaled by a positive

constant c , and therefore, the larger probability values will still be assigned to them. Thus, even though the probability value is not the same after scaling by $c > 0$, the label corresponding to the largest probability value will remain the same. These properties are required to obtain equivalent networks as discussed below.

Let $\bar{c} \in \mathbb{R}^{d_r+1}$ have all its coordinates equal to a real number c . Define

$$\tilde{A} := (A_{*,1} \quad \cdots \quad A_{*,j} + \bar{c} \quad \cdots \quad A_{*,d_r}) \quad \text{and} \quad \tilde{b} := b + \bar{c},$$

where $A_{*,k}$ denotes the k -th column of A , i.e., $\tilde{A}$ is the matrix A with $\bar{c}$ added to its j -th column. Then,

$$z(Ay + \tilde{b}) = z(Ay + b + \bar{c}) = z(Ay + b),$$

where the second equality holds since $\bar{c}$ translates the output of the DNN. Also,

$$z(\tilde{A}y + b) = z(Ay + b + y_j \bar{c}) = z(Ay + b),$$

where the first equality holds because $(\tilde{A}y)_i = \sum_k A_{i,k} y_k + y_j c$, and the second because $y_j \bar{c}$ translates the output of the DNN. Finally, when $c > 0$,

$$z((cA)y + cb) = z(c(Ay + b)) = z(Ay + b),$$

where the second equality holds because $c > 0$.

Any of the transformations above to the parameters of the output layer yield an equivalent network. Namely, we can translate any of the d_r columns in the matrix A , translate the bias vector b or scale A and b by a positive constant c . Moreover, any combination of these transformations also yields an equivalent network. To see this, let c be a positive real number and define

$$\tilde{A} := (A_{*,1} + \bar{c}_1 \quad \cdots \quad A_{*,k} + \bar{c}_k \quad \cdots \quad A_{*,d_r} + \bar{c}_{d_r}) \quad \text{and} \quad \tilde{b} := b + \bar{c}_b,$$

where each vector $\bar{c}_j$, $j = 1, \dots, d_r$, and $\bar{c}_b$ has the same value in all its coordinates. Then

$$\begin{aligned} z((c\tilde{A})y + c\tilde{b}) &= z\left(c\left(Ay + \sum_{j=1}^{d_r} y_j \bar{c}_j\right) + c(b + \bar{c}_b)\right) \\ &= z\left(c(Ay + b) + c\left(\sum_{j=1}^{d_r} y_j \bar{c}_j + \bar{c}_b\right)\right) \\ &= z(c(Ay + b)) = z(Ay + b). \end{aligned}$$

That is, we have $d_r + 2$ *degrees of freedom* to generate equivalent networks while recovering the output layer.

3.2 Single-Output Deep Neural Networks

In this case, the DNN makes a decision between two classes: C0 and C1. If the input x belongs to C0, $f(x) < 0$, whereas $f(x) > 0$ when x belongs to C1. With a transition point x , we have $Ay + b = 0$, where $y = f_{1..r}(x)$. Notice that, when the DNN has one output, A is a vector of length d_r and b is a scalar. Therefore, $Ay + b = 0$ can be regarded as an equation where the coordinates of A and b are the unknown values. With d_r transition points x_k such that $y_k = f_{1..r}(x_k)$ are linearly independent, the system of linear equations

$$Ay_k = -b, \quad k = 1, \dots, d_r,$$

has a unique solution. We do not know the value of b , however, the system above is equivalent to

$$b^{-1}Ay_k = -1, \quad k = 1, \dots, d_r.$$

Thus, by solving this new system we recover $\hat{A}^{(r+1)} = b^{-1}A$ and $\hat{b}^{(r+1)} = 1$.

As discussed in Section 3.1, an equivalent network is recovered if we multiply A and b by a positive constant. In this case, we multiply by b^{-1} . If $b > 0$, we recover an equivalent network. When $b < 0$, the network is not equivalent and it will classify the inputs incorrectly. In order to verify the correctness of the recovered parameters, we simply compare the output of the original DNN with the output of the recovered network; if the outputs are different, we flip the signs of $\hat{A}^{(r+1)}$ and $\hat{b}^{(r+1)}$.

This method requires finding transition points and solving a system of linear equations. We find transition points with the algorithm in [6], which requires polynomial number of queries and has polynomial time complexity. Solving the system is done in polynomial time and requires no additional queries.

3.3 Multiple-Output Deep Neural Networks

The idea to recover the parameters of the output layer is again to solve a system of linear equations. We will construct the system such that all the weights and biases in that layer are the unknown variables.

Let x be a transition point between classes i and j , where $1 \leq i, j \leq d_{r+1}$, and let $y = f_{1..r}(x)$. Then, $A_i y + b_i = A_j y + b_j$, which is equivalent to

$$A_i y + b_i - A_j y - b_j = 0.$$

We can extend the equation above to include all parameters in the output layer:

$$A_1 \mathbf{0} + 0b_1 + \dots + A_i y + b_i + \dots - A_j y - b_j + \dots + A_{d_{r+1}} \mathbf{0} + 0b_{d_{r+1}} = 0, \quad (5)$$

where $\mathbf{0}$ is the zero vector in $\mathbb{R}^{d_{r+1}}$. If we collect the corresponding expression (5) from various transition points, we create a system of equations with $d_{r+1}(d_r + 1)$ variables which can be written as

$$\underbrace{C(A_1, b_1, A_2, b_2, \dots, A_{d_{r+1}}, b_{d_{r+1}})}_A)^T = (0, \dots, 0)^T, \quad (6)$$

where C is a matrix whose rows are defined by the coefficients from Equation (5). If we obtained $d_{r+1}(d_r + 1)$ linearly independent equations, we would find a unique solution, however, that is not possible. Let $\tilde{A}_i$ denote (A_i, b_i) , thus $\mathbf{A} = (\tilde{A}_1, \dots, \tilde{A}_{d_{r+1}})$. By Equation (5), each row in C can be written as

$$(y, 1) \underbrace{(\mathbf{0} \cdots \tilde{\mathbf{I}} \cdots -\tilde{\mathbf{I}} \cdots \mathbf{0})}_{\tilde{\mathbf{I}}_{i,j}},$$

where $\tilde{\mathbf{I}}_{i,j}$ is a block matrix of size $(d_r + 1) \times d_{r+1}(d_r + 1)$ whose i -th and j -th blocks are the only non-zero blocks and $\tilde{\mathbf{I}}$ is the identity matrix of size $(d_r + 1)$. Then, the rows of C are in the span of

$$\begin{matrix} & 1 & 2 & 3 & \cdots & d_{r+1} \\ \begin{matrix} 1 \\ 2 \\ \vdots \\ d_{r+1} - 1 \end{matrix} & \begin{pmatrix} \tilde{\mathbf{I}} & -\tilde{\mathbf{I}} & \mathbf{0} & \cdots & \mathbf{0} \\ \tilde{\mathbf{I}} & \mathbf{0} & -\tilde{\mathbf{I}} & \cdots & \mathbf{0} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \tilde{\mathbf{I}} & \mathbf{0} & \mathbf{0} & \cdots & -\tilde{\mathbf{I}} \end{pmatrix} \end{matrix}.$$

This matrix has $d_{r+1}(d_r + 1) - (d_r + 1)$ rows, $d_{r+1}(d_r + 1)$ columns and has full rank $d_{r+1}(d_r + 1) - (d_r + 1)$. Moreover, by (6), C is in the kernel of $\mathbf{A}$ and since $\dim(\text{Im}(\mathbf{A})) = 1$, the maximal rank that C can have is $d_{r+1}(d_r + 1) - (d_r + 2)$.

To recover the parameters of the output layer, we gather $d_{r+1}(d_r + 1) - (d_r + 2)$ linearly independent equations (5) and solve the system (6). Notice that C not having full rank implies that there are multiple solutions; they represent the parameter values that yield different equivalent networks. Alternatively, we can fix a value for $d_r + 2$ variables and solve the system to obtain a unique solution. By fixing the value of the variables, we are choosing one among the many networks that are equivalent to the original one.

Similarly to the single-output case, this method entails finding transition points and solving a system of linear equations, therefore, its complexity is also polynomial in both, number of queries and time complexity.

4 Efficient Sign-Recovery in Contractive-Enough Layers

A layer is *contractive* when the number of inputs to that layer is larger than the number of neurons it contains; in other words, layer i is contractive when $d_{i-1} > d_i$. Carlini et al. show in [7, §4.2] a technique to determine the correct sign of the neurons in the first layer when it is contractive; as in [5], we will refer to this technique as *freeze*. Also in [5], the authors introduce the *SOE* sign recovery technique, which is applicable when consecutive layers are “contractive-enough”. In Section 4.1 we first discuss the applicability of freeze for deeper layers in the raw-output setting. Then, we present a version of freeze and SOE for the hard-label setting in Section 4.2 and Section 4.3, respectively.

4.1 Freezing Deeper Layers in the Raw-Output Setting

Assume the first layer to be contractive and that we already recovered $\hat{A}^{(1)}$ and $\hat{b}^{(1)}$. We want to determine the correct sign of the j -th neuron in that layer, denoted by η . The freeze technique in [7] works as follows. Let

$$\hat{f}_1(x) = \hat{A}^{(1)}x + \hat{b}^{(1)}$$

and x^* be a critical point for η ; then the j -th coordinate of $\hat{h} = \hat{f}_1(x^*)$ is exactly 0. We compute $x_+, x_- \in \mathbb{R}^{d_0}$ such that $\hat{f}_1(x_+) = \hat{h} + \varepsilon e_j$ and $\hat{f}_1(x_-) = \hat{h} - \varepsilon e_j$, where ε is a small positive real number and e_j is the j -th standard basis vector in $\mathbb{R}^{d_1}$. Notice that, regardless of the correctness of the recovered signs, we have the same behaviour with the original DNN: the j -th coordinate of $h = f_1(x^*)$ is 0 and f_1 evaluated at x_+ and x_- will only change the j -th coordinate of h (i.e., x_+ and x_- freeze the value before the ReLU of all neurons in the layer except η). Thus, the correct sign of η can be determined by observing whether $f(x_+) = f(x^*)$ or $f(x_-) = f(x^*)$. If $f(x_-) = f(x^*)$ holds, the recovered sign of η is correct, because $\sigma(h - \varepsilon e_j) = \sigma(h)$. However, if $f(x_+) = f(x^*)$ holds, we invert the sign of the j -th row of $\hat{A}^{(1)}$ and $\hat{b}_j^{(1)}$.

The freeze technique entails finding pre-images x_+ of $h + \varepsilon e_j$ and x_- of $h - \varepsilon e_j$ under $\hat{f}_1$, for all $j \in \{1, \dots, d_1\}$. Since no ReLUs are involved in $\hat{f}_1$, we can compute pre-images when the $d_1 \times d_0$ matrix $A^{(1)}$ has rank d_1 . This holds with very high probability when $d_0 > d_1$, i.e., the first layer is contractive. Freeze may be applicable in deeper layers as well. Daniely and Granot [9] employ freeze when recovering the second layer of a three-layer network. We discuss below the conditions to apply freeze in deeper layers.

We want to recover the signs for layer $i > 1$. Assume we have recovered $\hat{A}^{(i)}$ and $\hat{b}^{(i)}$. Let $F_{i-1}(x') = F_x^{(i-1)}x' + b_x^{(i-1)}$ be the map representing all $i-1$ previously recovered layers when the inputs $x' \in \mathbb{R}^{d_0}$ are restricted to the linear neighbourhood of $x \in \mathbb{R}^{d_0}$ (see Equation (4)). Also, let

$$\hat{f}_i(x') = \hat{A}^{(i)}F_{i-1}(x') + \hat{b}^{(i)} = \hat{A}^{(i)}F_x^{(i-1)}x' + (\hat{A}^{(i)}b_x^{(i-1)} + \hat{b}^{(i)}). \quad (7)$$

Then, we can apply freeze using $\hat{f}_i$ instead of $\hat{f}_1$. In this case, however, each critical point x^* for a neuron in layer i will induce a different map F_{i-1} . Thus, for each neuron, we first obtain $F_{x^*}^{(i-1)}, b_{x^*}^{(i-1)}$ and then compute x_+ and x_- to determine the sign. The pre-images can be computed if the matrix $\hat{A}^{(i)}F_x^{(i-1)}$ has rank d_i . This holds when the $d_i \times d_{i-1}$ matrix $A^{(i)}$ has rank d_i (implying $d_{i-1} \geq d_i$) and $\text{rank}(F_x^{(i-1)}) \geq d_i$. Almost always, the $d_{i-1} \times d_0$ matrix $F_x^{(i-1)}$ does not have full rank due to the ReLUs in layers 1 to $i-1$. Therefore, we require $d_{i-1} > d_i$ in practice. Additionally, ε must be small enough so that x_+ and x_- do not toggle any neuron in layers 1 to i except the target neuron.

4.2 Freeze in the Hard-Label Setting

Freeze requires access to the raw output of the DNN to check whether $f(x_+) = f(x^*)$ or $f(x_-) = f(x^*)$. However, we cannot access that information in the

hard-label setting, we only receive the most likely class label as the response to our queries. Here we present a version of freeze for the hard-label setting.

Let us focus on the first layer. Again, assume we already recovered $\hat{A}^{(1)}, \hat{b}^{(1)}$ and we want to determine the correct sign of η , the j -th neuron in that layer. Let $x \in \mathbb{R}^{d_0}$ be a transition point (between any two classes) that is not a dual point. If the first layer is contractive, we can compute $x' \in \mathbb{R}^{d_0}$ such that $\hat{f}_1(x') = \hat{h} + \varepsilon e_j$, where $\hat{h} = \hat{f}_1(x)$ and ε is a small real number. Unlike the raw-output case, the j -th coordinate of $\hat{h}$ is not 0. Let us consider the state of η in the original DNN when we choose ε such that η has the same state with x and x' :

- When η is inactive, the output of the first layer is the same with x and x' . Therefore, $f(x') = f(x)$ and x' is another transition point in the same decision boundary as x ; see Figure 3a.
- When η is active, the output of the first layer is different with x and x' . With very high probability, x' is not in the decision boundary; see Figure 3b. x' remains in the decision boundary when the vector $x' - x$ is parallel to that hyperplane.

We do not know the state of η with input x , but we can determine it as follows. Take a random direction Δ and compute $f(x' + \Delta)$ and $f(x' - \Delta)$. Consider Figure 3a again when the neuron is inactive; $f(x')$ is in the decision boundary, therefore, evaluating at $x' + \Delta$ and $x' - \Delta$ will result in different labels. On the other hand, when η is active, $f(x')$ is outside the decision boundary and $x' + \Delta$ and $x' - \Delta$ will result in the same labels; see Figure 3b. For the latter to hold, the magnitude of Δ should be sufficiently small not to cross the decision boundary and incorrectly determine the state of the neuron.

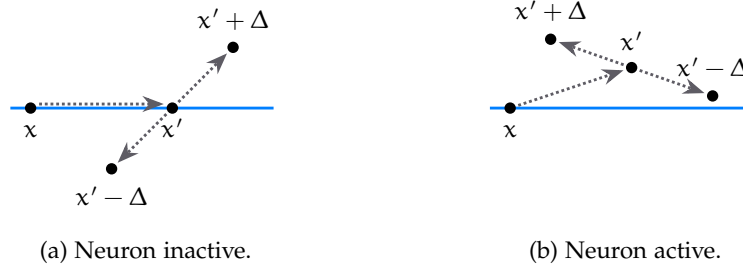


Fig. 3: Idea behind determining the state of the target neuron.

Knowing the state of η in the original DNN allows us to directly check whether the recovered sign is correct or not. We can also employ freeze in the hard-label case for deeper layers $i > 1$ using $\hat{f}_i$, as defined in Equation (7), instead of $\hat{f}_1$. The requirements to apply freeze in the hard-label setting are almost the same as in the raw-output case. For the first layer, we can use freeze

if $\text{rank}(A^{(1)}) = d_1$. For layers $i > 1$, we can apply it when (i) $\text{rank}(F_x^{(i-1)}) \geq d_i$, where x is a transition point between any two classes and (ii) ε is small enough so that x' does not toggle any neuron in layers 1 to i . Notice that condition (ii) is slightly different compared to the raw-output case.

Let t_T and q_T denote, respectively, the polynomial time and polynomial query complexity to find a transition point with the algorithm in [6]. To apply this method, we find a transition point x , then we find x' in polynomial time t_1 using linear algebra operations and we query the oracle on inputs $x + \Delta$ and $x - \Delta$. These steps are performed per neuron. In total, this method has polynomial time complexity $d_i(t_T + t_1)$ and query complexity $d_i(q_T + 2)$.

4.3 SOE in the Hard-Label Setting

Unlike the freeze technique, with SOE we can simultaneously recover the sign of all neurons in the target layer. Once more, assume that we have already recovered $\hat{A}^{(i)}$ and $\hat{b}^{(i)}$. Recall that when we restrict DNN inputs to be in the linear neighbourhood of a point x , we can collapse contiguous layers and we can observe the effect of a small change at the input of the network (see Equation (3)). Assume the DNN has one output only; if it has multiple outputs, we can simply ignore all but one. Then, the effect of a change Δ_k can be written as

$$\underbrace{G_x^{(i+1)} I_x^{(i)}}_c \underbrace{A^{(i)} F_x^{(i-1)}}_{y_k} \Delta_k^T = \underbrace{f(x + \Delta_k) - f(x)}_{z_k}.$$

With the recovered weights, we can obtain the value of y_k , however, we do not know the value of c . Additionally, in the raw-output setting we have access to the raw output the network and therefore the value z_k is known. Thus, the equation above can be rewritten as

$$c \cdot y_k = z_k,$$

which is an equation on the variables c . If we collect d_i linearly independent equations, we can solve for c and learn the state of each neuron: the j -th neuron is inactive if $c_j = 0$ and active otherwise. Knowing the state of the neurons allows us to recover the correct sign.

In the hard-label setting, we no longer have access to the value of z_k . However, the SOE method can still be applied. Let x be a transition point between classes α and β . Also, let $G_{x,\alpha}^{(i+1)}$ and $G_{x,\beta}^{(i+1)}$ be the rows of the matrix $G_x^{(i+1)}$ corresponding to the outputs of classes α and β , respectively. We choose Δ_k such that $x + \Delta_k$ is also a transition point between classes α and β and is in the linear neighbourhood of x . Then,

$$\begin{aligned} G_{x,\alpha}^{(i+1)} I_x^{(i)} A^{(i)} F_x^{(i-1)} \Delta_k^T &= f_\alpha(x + \Delta_k) - f_\alpha(x), \\ G_{x,\beta}^{(i+1)} I_x^{(i)} A^{(i)} F_x^{(i-1)} \Delta_k^T &= f_\beta(x + \Delta_k) - f_\beta(x), \end{aligned}$$

where f_α and f_β denote the DNN output corresponding to classes α and β , respectively. Subtracting the first equation above from the second, we get

$$\left(G_{x,\beta}^{(i+1)} - G_{x,\alpha}^{(i+1)} \right) I_x^{(i)} A^{(i)} F_x^{(i-1)} \Delta_k^T = f_\beta(x + \Delta_k) - f_\alpha(x + \Delta_k) - (f_\beta(x) - f_\alpha(x)).$$

Since x and $x + \Delta_k$ are transition points for the same classes, $f_\beta(x) - f_\alpha(x) = 0$ and $f_\beta(x + \Delta_k) - f_\alpha(x + \Delta_k) = 0$, therefore

$$\underbrace{\left(G_{x,\beta}^{(i+1)} - G_{x,\alpha}^{(i+1)}\right)}_c \underbrace{I_x^{(i)} A^{(i)} F_x^{(i-1)} \Delta_k^T}_{y_k} = 0.$$

Thus, we can collect d_i linearly independent equations and solve that system to recover the sign of the neurons.

When we recover $\hat{A}^{(i)}$ using the technique in [6], we also obtain the normal vector m to the decision boundary between classes α and β (see Figure 2). Since x and $x + \Delta_k$ are transition points in that decision boundary, both are orthogonal to m , therefore, $\langle m, \Delta_k \rangle = 0$, where $\langle \cdot, \cdot \rangle$ denotes the dot product. Notice that we want to find a vector c such that $c A^{(i)} F_x^{(i-1)} \Delta_k^T = 0$. Let $(A^{(i)} F_x^{(i-1)})^{-1}$ denote the pseudo-inverse of $A^{(i)} F_x^{(i-1)}$. If we set $c = m(A^{(i)} F_x^{(i-1)})^{-1}$,

$$c A^{(i)} F_x^{(i-1)} \Delta_k^T = m \Delta_k^T = 0$$

and we obtain a solution to the system of equations.

To construct the system of equations, we must find $d_i + 1$ transition points (namely, x and $x + \Delta_k$, for $k = 1, \dots, d_i$) and then solve a system of equations in polynomial time t_1 . Again, let t_T and q_T denote, respectively, the polynomial time and query complexity for finding a transition point. Then, hard-label SOE has polynomial time complexity $(d_i + 1)t_T + t_1$ and polynomial query complexity $(d_i + 1)q_T$. When we use the normal vector m , we require no oracle calls and the polynomial time complexity t is given by the linear algebra operations required to compute the pseudo-inverse of $A^{(i)} F_x^{(i-1)}$ and a vector-matrix multiplication.

5 Experiments and Conclusions

In this section, we present our experiments to evaluate the parameter recovery technique for the output layer and the sign-recovery methods. We first explain our choices for target networks in Section 5.1. Then, in Sections 5.2 and 5.3, respectively, we show our experimental results for the CIFAR10 network in [6] and our own network. Finally, conclusions from this work are in Section 5.4.

The experiments are executed using our non-optimised Python implementation. This implementation mainly uses the `numpy` and `scipy` libraries, and takes as a base the code from [5,6]. Our software implementation is available in <https://github.com/deividafonso281/hard-label-contract-output>.

5.1 Choice of target architectures

Here, we will denote the architecture of a network as $d_0 - d_1 - \dots - d_{r+1}$, where d_0 and d_{r+1} are, respectively, the length of the inputs and outputs of the DNN, and d_i , $1 \leq i \leq r$, is the number of neurons in layer i . If k successive layers

have the same number of neurons d , we will group them together as $d \times k$, e.g., $30 - 20 \times 3 - 10$ denotes $30 - 20 - 20 - 20 - 10$.

Given that our output-layer technique together with the methods in [6] would enable a full hard-label attack, we decided to use the same CIFAR10 network⁴ in [6]. This network has the following architecture: $3072 - 256 \times 3 - 64 - 10$. All neurons use ReLU as activation function and the output layer uses softmax.

Recall that our new sign-recovery methods are applicable when the number of neurons in the target layer i is lower than the rank of the transformation $F_x^{(i-1)}$ given by a transition point x and the previously recovered layers (see Section 4). Thus, our methods should be applicable to the first and fourth layers of the network above. However, we could apply them to the first layer only since the rank condition was not met when recovering signs in the fourth layer; we did not investigate why that network presented this behaviour. Instead, we trained a new network whose layers always contract, specifically, its architecture is $3072 - 1000 - 350 - 100 - 30 - 10$. This new network allowed us to successfully apply our sign recovery techniques. We trained this network on the CIFAR10 dataset and used the same training strategy as in [13,5,6]. More precisely, we used stochastic gradient descent with momentum equal to 0.9 for the training method, sparse categorical cross entropy for the loss function, batch size equal to 64 and learning rate 0.001.

5.2 Experimental Results - Network From [6]

The main goal was to test the recovery of the output layer. For this, we assume that we have already extracted the parameters of all layers. The first step is to find transition points to create the matrix C with the largest possible rank. In this case, $d_r = 64$ and $d_{r+1} = 10$, therefore, the maximum rank of C is $10(64 + 1) - (64 + 2) = 584$ (see Section 3.3). The experiments required about 6 hours and 30 minutes to build the system of equations. Next, as discussed in Section 3.3, we fixed the value of 66 parameters in the output layer. Particularly, we heuristically chose to set the 64 weights and bias of the first output to 0, and the first weight of the second output to 1, i.e., $\hat{A}_1^{(5)} = (0, \dots, 0)$, $\hat{b}_1^{(5)} = 0$ and $\hat{A}_{2,1}^{(5)} = 1$. Finally, solving the system of equations took a fraction of a second. To validate the accuracy of the recovered parameters, we sampled 5000 input vectors whose coordinates were taken uniformly at random from $(-1, 1)$. Then, we counted the number of inputs for which the output of the original and recovered networks matched. This was the case for all inputs.

We also applied our sign-recovery techniques. Freeze requires computing the pre-image x' of $\hat{h} + \varepsilon e_j$ and the evaluation of $x' + \Delta$ and $x' - \Delta$ (see Section 4.2). Using $\varepsilon = 10^{-5}$ and $\|\Delta\| = 10^{-9}$ in the experiments, we successfully recovered the sign for all 256 neurons in the first layer. The total execution time was 33.41 seconds, which is $6.5\times$ faster than the 220 seconds reported in [6]

⁴ Available in <https://github.com/jchavezsaab/hard-label-dnn-extraction>.

to recover a single batch of 64 signs with a parallel implementation (in total $220 \times 4 = 880$ seconds for the whole layer).

Recall that to get the parameters of the current target layer, the technique by Carlini et al. computes the normal vector $\mathbf{m}$ of the decision boundary. To test hard-label SOE, we executed first an experiment that does not use $\mathbf{m}$ and then a second experiment using this vector. In the first SOE experiment, the execution time was 358.27 seconds, which is slower than freeze, but still faster than the result in [6]. The whole time is practically spent finding transition points that yield a full-rank system. Due to some signs recovered wrongly, we executed the experiment multiple times to correctly recover the sign of all neurons. We attribute the wrong signs to precision errors when finding the critical points $\mathbf{x} + \Delta_{\mathbf{k}}$. Our experiments are performed with 64-bit floating point arithmetic, therefore, not exact. Also, as in [6], we know that in practice we cannot obtain transition points exactly in the decision boundary, but ϵ away from it. In our experiments, we estimated this distance to be 10^{-13} . On the other hand, we observed that re-executing the experiment still produced a few wrong signs, but different from those in previous executions. This is different from the *difficult neuron* behaviour [5], where the recovered sign for some neurons is consistently wrong. Therefore, we executed the experiment multiple times, 10 in our case, and decided the sign as the one with the highest frequency. This allowed correct sign recovery in the less realistic SOE experiment.

Since we obtain the signatures with the technique by Carlini et al., we already know the normal vector $\mathbf{m}$. We can think that the work required in our first SOE experiment is already done by the signature-recovery procedure. This procedure also requires searching for transition points to get a basis of the decision boundary and in turn, the critical hyperplanes; as reported in [6], signature recovery is the most time-consuming step. Thus, the second SOE experiment is the more realistic attack scenario. In this case, we simply compute the pseudo-inverse of a matrix and perform a matrix-vector multiplication (see Section 4.3). The execution time is under a second and, therefore, the best option available.

5.3 Experimental Results - Our Network

The main goal of the experiments with our network was to test the sign recovery techniques for deeper layers. We obtained successful results with freeze for all layers 1 to 4 using the same values $\epsilon = 10^{-5}$ and $\|\Delta\| = 10^{-9}$ as in Section 5.2. The execution times are in Table 1. We also executed two experiments with SOE, the first one solving the system of linear equations and the second one using the normal vector $\mathbf{m}$. The results are also in Table 1. Again, the less realistic scenario requires sensibly more time to find the transition points and construct the system of equations. The more realistic scenario is practically instantaneous.

Finally, we recovered the parameters of the output layer. In this case, $d_r = 30$ and $d_{r+1} = 10$, therefore, the maximum rank of $\mathbf{C}$ is $10(30 + 1) - (30 + 2) = 278$ and it took about 13 hours to build the system of equations. As in

Method	Time (in seconds)			
	Layer 1	Layer 2	Layer 3	Layer 4
Freeze	131.79	50.87	18.78	9.15
SOE (no m)	1457.88	516.65	150.59	47.35
SOE (using m)	0.50	0.30	0.23	0.22

Table 1: Time to recover all neuron signs in layers 1 to 4 of our network.

the experiment with the other network, we fixed $\hat{A}_1^{(5)} = (0, \dots, 0)$, $\hat{b}_1^{(5)} = 0$, $\hat{A}_{2,1}^{(5)} = 1$ and solving the system of equations took a fraction of a second. The time to build the system of equations was higher due to some neurons being rarely active, therefore, we required to test a large amount of points; this situation is not rare and is also reported in [6].

5.4 Conclusions

We continued the study of parameter extraction of ReLU-based DNNs in the hard-label setting when the attacker exclusively has oracle access to the network. The recent novel attack by Carlini et al. demonstrated how to efficiently recover the parameters for all layers except the output layer. Here, we introduced a new method to extract this missing layer. Thus, a full hard-label recovery attack is now possible. Additionally, we presented new sign-recovery techniques applicable on contractive-enough layers. We showed experimentally that these techniques can have a big impact in real execution time.

One natural future direction is to develop an end-to-end implementation of the full hard-label attack. Also, to reduce the real time complexity, we need more efficient algorithms to obtain the signature from transition points. Other possible lines of research are to study extraction attacks for networks using activation functions that are not piece-wise linear and to address the seemingly more challenging problem of extraction attacks on other machine learning architectures (e.g., transformers).

Acknowledgments. We would like to express our sincere gratitude to Adi Shamir for valuable ideas and discussions which helped shape this work and Francisco Rodríguez-Henríquez for insightful feedback. Finally, we thank the reviewers for their helpful comments and suggestions.

References

1. Baum, E.B.: A Polynomial Time Algorithm That Learns Two Hidden Unit Nets. *Neural Computation* 2(4), 510–522 (12 1990). <https://doi.org/10.1162/neco.1990.2.4.510>

2. Baum, E.B.: Neural net algorithms that learn in polynomial time from examples and queries. *IEEE Transactions on Neural Networks* 2(1), 5–19 (1991). <https://doi.org/10.1109/72.80287>
3. Blum, A.L., Rivest, R.L.: Training a 3-node neural network is NP-complete. *Neural Networks* 5(1), 117–127 (1992). [https://doi.org/10.1016/S0893-6080\(05\)80010-3](https://doi.org/10.1016/S0893-6080(05)80010-3)
4. Canales-Martínez, I.A., Chávez-Saab, J., Hambitzer, A., Rodríguez-Henríquez, F., Satpute, N., Shamir, A.: Polynomial Time Cryptanalytic Extraction of Neural Network Models. *Cryptology ePrint Archive*, Paper 2023/1526 (2023), <https://eprint.iacr.org/2023/1526>
5. Canales-Martínez, I.A., Chávez-Saab, J., Hambitzer, A., Rodríguez-Henríquez, F., Satpute, N., Shamir, A.: Polynomial Time Cryptanalytic Extraction of Neural Network Models. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology – EUROCRYPT 2024*. *Lecture Notes in Computer Science*, vol. 14653, pp. 3–33. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-58734-4_1
6. Carlini, N., Chávez-Saab, J., Hambitzer, A., Rodríguez-Henríquez, F., Shamir, A.: Polynomial Time Cryptanalytic Extraction of Deep Neural Networks in the Hard-Label Setting. In: Fehr, S., Fouque, P.A. (eds.) *Advances in Cryptology – EUROCRYPT 2025*. *Lecture Notes in Computer Science*, vol. 15601, pp. 364–396. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-91107-1_13
7. Carlini, N., Jagielski, M., Mironov, I.: Cryptanalytic Extraction of Neural Network Models. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology – CRYPTO 2020*. *Lecture Notes in Computer Science*, vol. 12172, pp. 189–218. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-56877-1_7
8. Chen, Y., Dong, X., Guo, J., Shen, Y., Wang, A., Wang, X.: Hard-Label Cryptanalytic Extraction of Neural Network Models. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. *Lecture Notes in Computer Science*, vol. 15491, pp. 207–236. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0944-4_7
9. Daniely, A., Granot, E.: An Exact Poly-Time Membership-Queries Algorithm for Extracting a Three-Layer ReLU network. In: *The Eleventh International Conference on Learning Representations, ICLR 2023*. OpenReview.net (2023), <https://openreview.net/forum?id=-CoNloheTs>
10. Fefferman, C.: Reconstructing a neural net from its output. *Revista Matemática Iberoamericana* 10(3), 507–555 (1994), <http://eudml.org/doc/39464>
11. Foerster, H., Mullins, R., Shumailov, I., Hayes, J.: Beyond Slow Signs in High-fidelity Model Extraction. In: Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., Zhang, C. (eds.) *Advances in Neural Information Processing Systems*. vol. 37, pp. 19496–19522. Curran Associates, Inc. (2024), https://proceedings.neurips.cc/paper_files/paper/2024/file/22ae669a35bb9e70eb93ab77c1eff5b4-Paper-Conference.pdf
12. Jagielski, M., Carlini, N., Berthelot, D., Kurakin, A., Papernot, N.: High Accuracy and High Fidelity Extraction of Neural Networks. In: Capkun, S., Roesner, F. (eds.) *Proceedings of the 29th USENIX Conference on Security Symposium*. pp. 1345–1362. SEC’20, USENIX Association, USA (2020)
13. Lin, Z., Memisevic, R., Konda, K.R.: How far can we go without convolution: Improving fully-connected networks (2015), <https://arxiv.org/abs/1511.02580>

14. Oliynyk, D., Mayer, R., Rauber, A.: I Know What You Trained Last Summer: A Survey on Stealing Machine Learning Models and Defences. *ACM Computing Surveys* 55(14s), 1–41 (7 2023). <https://doi.org/10.1145/3595292>